大模型技术栈面试题汇总

> *本文档整理了18张面试图片中的所有问题，按技术栈分类并提供详细解答*
> *生成日期：2026-03-21*

目录

1. Agent设计与架构
2. RAG检索增强生成
3. Multi-Agent多智能体系统
4. Transformer与注意力机制
5. 大模型训练与微调
6. Prompt工程与上下文工程
7. 工具调用与Function Calling
8. 向量数据库与知识库
9. 后端架构与中间件
10. 数据库
11. 缓存与Redis
12. 算法与数据结构
13. Python与Go语言
14. 计算机网络
15. 其他技术问题

一、Agent设计与架构

1.1 基础概念

Q1: 介绍auto deployer的实现难点

解答：

Auto Deployer（自动部署系统）的实现难点主要包括：

难点	说明	解决方案
环境一致性	开发/测试/生产环境差异	容器化（Docker）、基础设施即代码（IaC）
回滚机制	部署失败时快速恢复	蓝绿部署、金丝雀发布、版本标记
依赖管理	服务间依赖关系复杂	依赖图分析、按拓扑顺序部署
健康检查	如何判断部署成功	多维度健康检查（HTTP、TCP、自定义探针）
零停机部署	不影响线上服务	滚动更新、流量切换

Q2: 介绍motion graphic agent是什么，架构是什么

解答：

Motion Graphic Agent是一种能够自动生成动态图形/动画的AI Agent。

典型架构：

```
用户输入（文本/语音）
      ↓
意图理解模块 → 提取动画需求（风格、时长、元素等）
      ↓
素材检索模块 → 从素材库获取图片/图标/字体
      ↓
脚本生成模块 → 生成动画时间轴和关键帧
      ↓
渲染引擎 → 合成最终视频/GIF
      ↓
输出结果
```

Q3: 项目用的是把skill直接塞进系统提示词中，如果skill太多，占用的上下文窗口太大，怎么处理?

解答:

解决方案:

1. **Skill检索机制**
    - 将Skill描述向量化存储
    - 根据用户查询动态检索相关Skill
    - 只加载Top-K相关Skill到Prompt
2. **Skill分层管理**
    - 基础Skill：常驻内存
    - 领域Skill：按需加载
    - 临时Skill：单次会话有效
3. **Skill摘要压缩**
    - 使用LLM对Skill描述进行摘要
    - 保留核心功能描述，去除冗余信息
4. **Skill组合优化**
    - 将相关Skill合并为复合Skill
    - 减少独立Skill数量

Q4: 如果两个skill实际内容不一样，但是description相似，导致agent每次加载错skill，怎么解决？

解答：

解决方案：

1. **优化Skill描述**
    - 增加区分性关键词
    - 明确标注适用场景和边界条件
    - 添加负面示例（什么情况下不该用）
2. **引入Skill示例**
    - 为每个Skill提供使用示例
    - 通过Few-shot方式帮助模型理解
3. **多路召回+重排序**
    - 召回多个候选Skill
    - 使用更精确的模型进行二次筛选
4. **用户反馈机制**
    - 记录Skill选择准确率
    - 根据反馈持续优化描述

Q5: agent设计你觉得最重要的部分是什么？

解答：

上下文工程（**Context Engineering**）

原因：

- Agent的能力上限由上下文质量决定
- 好的上下文能让模型充分发挥能力
- 上下文管理直接影响成本和性能

Q6: 上下文工程你觉得有哪些要注意的点?

解答:

注意点	说明
信息筛选	只保留与当前任务相关的信息
信息排序	重要信息放在前面（LLM有位置偏见）
格式统一	使用结构化格式（JSON/XML）便于解析
长度控制	避免超出模型上下文限制
动态更新	根据对话进展更新上下文
记忆分层	区分短期记忆和长期记忆

Q7: 了解主流的agent设计吗?

解答:

主流**Agent**设计模式:

模式	特点	代表
ReAct	推理+行动交替	LangChain Agent
Plan-and-Execute	先规划后执行	Plan-and-Solve
Reflection	带自我反思	Reflexion
Multi-Agent	多Agent协作	AutoGen、CrewAI

Q8: gemini cli设计你觉得最重要的部分

解答:

Gemini CLI设计的核心要点:

1. 用户体验
    - 响应速度快

        - 输出格式清晰
        - 支持流式输出
    2. 上下文管理
        - 会话记忆
        - 文件上下文理解
    3. 工具集成
        - 代码执行
        - 文件操作
        - 网络请求
    4. 安全性
        - 沙箱执行
        - 权限控制

Q9: gemini cli你觉得可以优化的点

解答：

优化方向	具体建议
离线支持	支持本地模型，减少网络依赖
插件系统	允许用户扩展功能
成本透明	显示Token消耗和预估费用
版本控制	集成Git，追踪代码变更
团队协作	支持共享会话和配置

Q10: multi agent架构主流的实现方案

解答：

主流方案：

1. 中心化架构（**Orchestrator**）

        - 中央协调器分配任务
        - 适合任务明确的场景
    2. **去中心化架构（P2P）**
        - Agent间直接通信
        - 适合协作探索场景
    3. **层级架构（Hierarchical）**
        - 管理层+执行层
        - 适合复杂项目管理
    4. **市场架构（Market-based）**
        - Agent竞标获取任务
        - 适合资源分配场景

Q11: multi agent的交流是靠什么

解答：

通信机制：

机制	说明	适用场景
共享内存	共同访问状态存储	需要强一致性
消息队列	异步消息传递	解耦场景
直接调用	API直接调用	低延迟场景
事件总线	发布订阅模式	广播通知
黑板系统	共享工作空间	协作问题解决

Q12: Agent的记忆类型有哪些？如果上下文超过模型的最大长度，通常如何解决？

解答：

记忆类型：

类型	特点	存储方式
感官记忆	当前输入	内存
短期记忆	当前会话	内存/Prompt
工作记忆	活跃信息	向量数据库
长期记忆	历史知识	向量数据库/知识图谱
情景记忆	具体事件	数据库存储

上下文超限解决方案：

1. 滑动窗口：保留最近N轮
2. 摘要压缩：对历史进行LLM摘要
3. 分层检索：重要信息存入长期记忆
4. 选择性遗忘：丢弃低重要性信息

Q13: 除了ReAct模式之外，Agent还有哪些常见范式？

解答：

范式	核心思想	适用场景
ReAct	推理-行动交替	通用场景
Plan-and-Solve	先规划后执行	复杂多步任务
Reflexion	自我反思迭代	高质量输出
LATS	树搜索+反思	多方案探索
DSPy	程序化优化Prompt	系统优化
TOT	思维树	需要探索的任务

Q14: 是否使用过多智能体（Multi-Agent）？如何搭建多智能体系统？

解答：

搭建步骤：

1. 角色定义
    - 明确每个Agent的职责
    - 定义输入输出接口
2. 通信设计
    - 选择通信机制
    - 定义消息格式
3. 协调机制
    - 任务分配策略
    - 冲突解决机制
4. 状态管理
    - 共享状态设计
    - 状态同步机制

Q15: 常见的多智能体架构有哪些?

解答:

- **CrewAI**：角色扮演式协作
- **AutoGen**：对话式多Agent
- **LangGraph**：基于状态机的工作流
- **MetaGPT**：模拟软件公司流程
- **AgentScope**：阿里巴巴开源框架

Q16: Agent的"规划-执行-反思"闭环如何实现?

解答:

```
用户输入
    ↓
[规划阶段] → 生成执行计划
    ↓
[执行阶段] → 按计划调用工具/生成内容
    ↓
[反思阶段] → 检查结果是否符合预期
    ↓
    ├─ 符合 → 返回结果
    └─ 不符合 → 重新规划 → 循环
```

关键点：

- 设置最大迭代次数防止死循环
- 明确定义"完成"的判断标准
- 记录中间状态便于调试

Q17: 多轮对话中，记忆如何分级管理？缓存策略怎么设计？

解答：

分级管理：

级别	内容	存储	生命周期
L1	当前轮次	内存	单次请求
L2	当前会话	Redis	会话期间
L3	用户级记忆	向量DB	长期
L4	全局知识	知识库	永久

缓存策略：

- LRU：最近最少使用
- TTL：设置过期时间
- 重要性加权：重要信息优先保留

Q18: 如何设计一个生产级的AI Agent系统架构?

解答:

核心组件:

```
┌──────────────────────────────────────────┐
│                API Gateway                 │
│          (限流/认证/路由)                  │
└──────────────────────────────────────────┘
                  │
                  ↓
┌──────────────────────────────────────────┐
│            Agent Orchestrator              │
│       (任务分发/状态管理/监控)             │
└──────────────────────────────────────────┘
                  │
                  ↓
┌──────────────────────────────────────────┐
│            Agent Worker Pool               │
│       (多Agent实例/负载均衡)               │
└──────────────────────────────────────────┘
                  │
                  ↓
┌──────────────────────────────────────────┐
│      Memory Store     │   Tool Registry   │
│   (向量数据库)        │   (工具注册中心)  │
└──────────────────────────────────────────┘
```

Q19: 长期记忆和短期记忆在设计上有什么区别? 本质上有什么区别?

解答:

维度	短期记忆	长期记忆
存储位置	内存/Prompt	向量数据库
访问速度	快	慢（需检索）
容量	小（受上下文限制）	大
持久性	会话级	永久
更新频率	高	低
本质区别	工作空间	知识库

Q20: Agent的长记忆和短记忆之间，怎么做到协同工作的？两者的衔接逻辑是什么？

解答：

协同流程：

```
用户输入
    ↓
短期记忆检查 → 是否足够回答?
    ↓是                ↓否
直接回答      长期记忆检索
                  ↓
          相关性筛选
                  ↓
          融入短期记忆
                  ↓
            生成回答
                  ↓
          重要信息写入长期记忆
```

衔接逻辑：

- 触发机制：短期记忆不足时触发检索
- 优先级：短期记忆 > 高度相关的长期记忆
- 更新机制：重要信息从短期转入长期

Q21: 有什么思路能让自己做的Agent更智能?

解答:

1. **增强推理能力**
    - 引入Chain-of-Thought
    - 使用Self-Consistency
2. **提升工具使用**
    - 工具学习（Tool Learning）
    - 错误恢复机制
3. **优化记忆管理**
    - 更好的信息筛选
    - 知识图谱集成
4. **增加自我反思**
    - 输出质量评估
    - 迭代优化
5. **多模态能力**
    - 图像理解
    - 语音交互

Q22: 系统中的Agent包含哪些具体状态?

解答:

状态	说明
Idle	空闲等待
Planning	规划阶段
Executing	执行阶段
Waiting	等待外部输入
Reflecting	反思阶段
Completed	任务完成
Failed	执行失败
Paused	暂停状态

Q23: 后端Agent是否支持多租户同时调用?

解答:

支持方案:

1. 租户隔离
    - 数据隔离: 每个租户独立数据库/Schema
    - 资源隔离: 容器/进程级别隔离
2. 会话管理
    - 租户ID贯穿请求链路
    - 上下文按租户隔离存储
3. 资源限制
    - 按租户设置QPS限制
    - 按租户设置Token配额

Q24: Session和User ID是如何绑定的?

解答:

绑定方式:

```
用户登录 → 生成Session ID → 与User ID映射存储
                  ↓
        Redis/数据库维护映射关系
                  ↓
请求携带Session ID → 查询对应User ID → 获取用户上下文
```

存储结构：

- Key: Session ID
- Value: User ID + 会话上下文 + 过期时间

Q25: Agent工具调用的完整业务流程是怎样的？

解答：

```
1. 意图识别 → 判断是否需要调用工具
2. 工具选择 → 从可用工具中选择合适的
3. 参数提取 → 从用户输入提取工具参数
4. 参数校验 → 验证参数格式和范围
5. 工具执行 → 调用外部API或函数
6. 结果处理 → 解析工具返回结果
7. 结果整合 → 将结果融入回答
```

Q26: 什么是Agent的反思机制？对于心理咨询Agent，有没有让模型在回答前先检查一遍自己的语气是否专业？

解答：

反思机制：

Agent在生成最终回答前，先进行自我检查和评估的过程。

心理咨询Agent的反思维度：

维度	检查内容
语气	是否温和、共情
专业性	是否符合心理咨询伦理
安全性	是否识别出危机信号
准确性	建议是否有依据

实现方式：

- 使用独立的反思Prompt
- 设置检查清单（Checklist）
- 不通过则重新生成

Q27: Agent发生"工具调用"时，SSE推送的事件结构中通常包含哪些字段？

解答：

```
{
  "event": "tool_call",
  "data": {
    "tool_name": "search",
    "tool_input": {"query": "..."},
    "tool_output": {...},
    "timestamp": "2024-01-01T00:00:00Z",
    "execution_time": 1200,
    "status": "success"
  }
}
```

字段说明：

- `event`：事件类型
- `tool_name`：工具名称
- `tool_input`：输入参数

- `tool_output`：输出结果
- `timestamp`：时间戳
- `execution_time`：执行耗时
- `status`：执行状态

Q28: Agent如何与MCP Server连接通信？

解答：

MCP（Model Context Protocol）通信流程：

```
Agent ←→ MCP Client ←→ MCP Server ←→ 外部资源
```

连接方式：

1. **Stdio**：本地进程通信
2. **SSE**：服务器推送事件
3. **HTTP**：RESTful API

通信流程：

1. Agent发送上下文请求
2. MCP Client转发到Server
3. Server访问资源（文件/数据库/API）
4. 返回结构化上下文
5. Agent整合上下文生成回答

Q29: Agent一般在什么阶段去查询向量知识库？通过什么方式去查询？

解答：

查询时机：

阶段	说明
接收用户输入后	理解意图时检索相关背景知识
规划阶段	检索类似任务的执行方案
执行阶段	检索工具使用说明
生成阶段	检索事实信息确保准确性

查询方式：

- 向量化用户查询
- 相似度搜索（Cosine Similarity）
- 混合检索（向量+关键词）
- 重排序优化

Q30: 查询知识库的工具函数，其标准输入和输出是什么？

解答：

标准输入：

```
{
  "query": "用户查询文本",
  "top_k": 5,
  "filters": {
    "category": "技术文档",
    "date_range": ["2024-01-01", "2024-12-31"]
  },
  "embedding_model": "text-embedding-3"
}
```

标准输出：

```
{
  "results": [
    {
      "content": "检索到的文本内容",
      "source": "文档来源",
      "score": 0.95,
      "metadata": {...}
    }
  ],
  "total": 100,
  "query_time": 50
}
```

Q31: 如何追踪Agent的单次请求完整链路？跨多个服务的Trace怎么设计？

解答：

链路追踪设计：

```
用户请求 → Gateway
    ↓ [TraceID: abc123]
Agent服务
    ↓ [Span: intent_recognition]
工具调用服务
    ↓ [Span: tool_execution]
外部API
```

关键要素：

- **TraceID**: 全局唯一标识
- **SpanID**: 每个步骤的标识
- **ParentSpanID**: 父子关系
- **Timestamp**: 时间戳
- **Tags**: 业务标签

实现工具：

- OpenTelemetry
- Jaeger
- Zipkin

Q32: Agent系统的核心监控指标有哪些？怎么设计告警策略？

解答：

核心指标：

类别	指标	说明
性能	响应时间	P50/P95/P99
性能	Token消耗	输入/输出Token数
性能	吞吐量	QPS/TPS
质量	满意度	用户反馈评分
质量	任务成功率	成功完成比例
稳定性	错误率	各类错误占比
成本	单次请求成本	模型调用费用

告警策略：

- 响应时间P99 > 5s → 告警
- 错误率 > 1% → 告警
- Token消耗突增 → 告警

Q33: 高并发场景下，如何设计Agent服务的弹性伸缩策略？

解答：

伸缩策略：

1. **水平扩容**
    - 基于CPU/内存使用率
    - 基于请求队列长度
    - 基于响应时间
2. **垂直扩容**
    - 增加单实例资源配置
    - 优化模型推理效率
3. **智能路由**
    - 按复杂度分配不同模型
    - 简单任务用小模型
4. **缓存优化**
    - 相似请求结果缓存
    - 减少重复计算

Q34: Agent推理链路包含多个串行工具，响应延迟高，该如何优化?

解答:

优化方案:

方案	说明
并行化	无依赖的工具并行调用
预加载	预加载常用工具结果
异步化	非关键工具异步执行
缓存	缓存工具调用结果
模型优化	使用更快的模型处理简单步骤

Q35: 如何提升AI生成质量?

解答:

1. **提示词优化**
    - 使用Few-shot示例
    - 明确输出格式要求
    - 添加角色设定
2. **上下文增强**
    - RAG检索相关信息
    - 多轮对话保持连贯性
3. **后处理**
    - 事实核查
    - 格式校验
    - 敏感内容过滤
4. **模型选择**
    - 复杂任务用大模型
    - 简单任务用小模型

Q36: 什么是幻觉? 解决方案?

解答:

幻觉（Hallucination）：模型生成看似合理但实际错误的内容。

解决方案:

层级	方案
数据层	使用高质量训练数据
模型层	微调增强事实性
Prompt层	要求模型标注不确定性
检索层	RAG提供事实依据
后处理层	事实核查、人工审核

Q37: 如果大模型接入数据库，如何避免信息泄露或幻觉?

解答:

防护措施:

1. 权限控制
    - 数据库账号最小权限
    - 字段级访问控制
2. 查询限制
    - 只读权限
    - 敏感表脱敏
    - 查询结果行数限制
3. 输出审核
    - 敏感信息检测
    - 结果格式化（去除原始数据）
4. 审计日志
    - 记录所有查询
    - 异常行为监控

Q38: LLM的上下文和长短期记忆是怎么做的？

解答：

上下文管理：

```
当前Prompt = 系统提示 + 历史对话（最近N轮）+ 检索到的长期记忆 + 当前输入
```

记忆实现：

类型	实现方式
短期记忆	直接放在Prompt中
工作记忆	向量数据库存储，按需检索
长期记忆	结构化存储（知识图谱）

Q39: 详细讲一下流式输出原理/应用？

解答：

原理：

模型生成是逐个Token进行的，流式输出将每个Token实时推送给用户。

技术实现：

```
模型生成Token → Server-Sent Events(SSE) → 前端实时展示
```

优势：

- 用户体验好（无需等待）
- 可提前取消无用生成
- 适合长文本生成

应用场景：

- 聊天对话
- 代码生成
- 文章写作

Q40: 你在项目中是怎么处理agent的上下文过长的?

解答:

处理方法:

1. 滑动窗口
    - 保留最近N轮对话
    - 丢弃过期内容
2. 摘要压缩
    - 对历史对话进行LLM摘要
    - 保留关键信息
3. 分层存储
    - 重要事实存入长期记忆
    - 临时信息丢弃
4. 选择性保留
    - 保留关键决策点
    - 保留用户偏好

Q41: 如果让你设计一个能自动写sql的agent，你会怎么拆解?

解答:

模块拆解:

```
┌──────────────────────────────────────────┐
│  1. Schema理解模块                        │
│     - 解析数据库结构                       │
```

```
│      - 理解表关系                                  │
└──────────────────────────────────────────────────┘
                  │
                  ↓
┌──────────────────────────────────────────────────┐
│  2. 意图理解模块                                   │
│      - 解析用户自然语言需求                          │
│      - 提取查询条件                                 │
└──────────────────────────────────────────────────┘
                  │
                  ↓
┌──────────────────────────────────────────────────┐
│  3. SQL生成模块                                    │
│      - 生成SQL语句                                  │
│      - 支持复杂查询（JOIN/子查询）                    │
└──────────────────────────────────────────────────┘
                  │
                  ↓
┌──────────────────────────────────────────────────┐
│  4. 校验优化模块                                   │
│      - 语法检查                                     │
│      - 性能优化建议                                 │
└──────────────────────────────────────────────────┘
                  │
                  ↓
┌──────────────────────────────────────────────────┐
│  5. 执行反馈模块                                   │
│      - 执行SQL                                      │
│      - 格式化结果                                   │
└──────────────────────────────────────────────────┘
```

Q42: 你在项目中遇到的最难的bug是什么？

解答：

（这是一个开放性问题，示例回答）

案例：**LangChain内存泄漏**

现象：

- 服务运行一段时间后内存持续增长
- 最终导致OOM

排查过程:

1. 使用内存分析工具定位
2. 发现ConversationBufferMemory未清理
3. 历史消息无限累积

解决方案:

- 设置最大历史轮数限制
- 定期清理过期会话
- 使用弱引用管理内存

Q43: 怎么理解agent中的tool calling?

解答:

Tool Calling是Agent调用外部工具（函数/API）的能力。

核心流程:

```
用户输入 → Agent判断需要工具 → 选择工具 → 提取参数 → 执行工具 → 整合结果 → 生成回答
```

关键要素:

- 工具描述（让模型理解工具用途）
- 参数Schema（定义输入格式）
- 执行逻辑（实际调用代码）

Q44: 你了解哪些agent开发的框架？框架里有哪些组件？

解答：

主流框架：

框架	核心组件
LangChain	Chains、Agents、Memory、Tools、Retrievers
LlamaIndex	Index、Query Engine、Data Connectors
AutoGen	ConversableAgent、GroupChat、UserProxy
CrewAI	Agent、Task、Crew、Process
LangGraph	StateGraph、Node、Edge

Q45: memory是什么了解吗？

解答：

Memory是Agent存储和检索信息的能力。

类型：

- **Buffer Memory**：简单缓存历史消息
- **Vector Memory**：基于向量检索的记忆
- **Entity Memory**：提取和记忆实体信息
- **Conversation Knowledge Graph**：知识图谱形式存储

Q46: 上下文窗口满了如果需要压缩，怎么去压缩？如何筛选有效信息和垃圾信息，保证正确率最大？

解答：

压缩策略：

1. 摘要压缩
    - 使用LLM生成历史摘要
    - 保留关键决策和事实
2. 重要性评分
    - 基于语义重要性打分
    - 保留高分信息
3. 实体提取
    - 提取关键实体和关系
    - 丢弃冗余描述

筛选方法：

- 与当前查询的相关性
- 信息的新颖性（避免重复）
- 信息的时效性

Q47: RAG的流程是什么？有哪几种方式实现？

解答：

标准流程：

```
文档 → 分块 → Embedding → 向量存储
                                ↓
用户查询 → Embedding → 相似度检索 → 获取Top-K块 → 生成回答
```

实现方式：

方式	特点
Naive RAG	基础检索+生成
Advanced RAG	查询改写、重排序、递归检索
Modular RAG	模块化设计，灵活组合

Q48: 除了RAG你在项目中还用到了哪些Agent相关的技术?

解答:

- **Function Calling**：工具调用
- **Memory Management**：记忆管理
- **Multi-Agent Orchestration**：多Agent协调
- **Planning & Reflection**：规划与反思
- **Prompt Chaining**：提示链
- **Guardrails**：安全护栏

Q49: MCP了解吗？用过吗？底层是如何实现的?

解答:

MCP（Model Context Protocol）是Anthropic提出的开放协议，用于标准化AI模型与外部数据源的连接。

核心概念:

- **Server**：提供资源访问能力
- **Client**：连接Server的客户端
- **Resources**：可被访问的数据
- **Tools**：可被调用的功能

底层实现:

- 基于JSON-RPC通信
- 支持Stdio/SSE/HTTP传输
- 资源通过URI标识

Q50: Skill了解吗？用过吗？底层是如何实现的？比如渐进式披露是怎么实现的原理是什么？

解答：

Skill是Agent的可复用能力单元。

实现原理：

```
Skill = 描述 + 参数Schema + 执行函数
```

渐进式披露：

- 初始只提供Skill名称和简要描述
- 根据用户意图逐步展示详细信息
- 避免一次性加载过多信息

Q51: FastApi常用的包都了解吗？

解答：

常用包：

包名	用途
pydantic	数据验证和序列化
sqlalchemy	ORM数据库操作
alembic	数据库迁移
pytest	测试框架
httpx	异步HTTP客户端
redis-py	Redis客户端
celery	异步任务队列

Q52: OpenClaw用过吗？你觉得他的核心亮点是什么？

解答：

OpenClaw是一个开源的AI Agent框架。

核心亮点：

- 模块化设计
- 支持多模型后端
- 丰富的工具生态
- 可视化工作流编排

Q53: A2A协议知道吗？

解答：

A2A（Agent-to-Agent）协议是Google提出的Agent间通信协议。

核心特性：

- 标准化Agent间通信
- 支持能力发现和协商
- 安全认证机制
- 跨平台兼容

二、RAG检索增强生成

Q54: 什么是RAG？它是怎么提升生成质量的？

解答：

RAG（Retrieval-Augmented Generation）将外部知识检索与大模型生成相结合。

提升质量的方式：

方面	说明
减少幻觉	基于真实文档生成
知识时效	可接入最新数据
领域专业	接入专业知识库
可溯源	回答可追溯到来源

Q55: 与传统的检索+模型生成的流程有什么不同？

解答：

传统方式	RAG方式
检索和生成分离	检索结果直接用于增强生成
检索结果可能很长	会选择性使用检索内容
生成时不考虑检索	生成时以检索内容为基础

Q56: 在RAG系统中，检索模块通常如何设计？

解答：

检索模块组成：

```
查询理解 → 查询改写 → 多路召回 → 重排序 → 结果筛选
```

关键技术：

- 混合检索（向量+关键词）
- 查询扩展
- 重排序模型（Cross-Encoder）

Q57: 如何评估一个RAG系统是否有效?

解答:

评估指标:

指标	说明
检索准确率	检索到的文档是否相关
回答准确率	生成内容是否正确
召回率	是否找到所有相关信息
F1分数	综合准确率和召回率
延迟	检索+生成耗时

Q58: 如果发现回答仍然存在hallucination，你会如何排查问题?

解答:

排查步骤:

1. 检查检索结果
    - 检索到的文档是否包含正确答案
    - 检索结果是否被正确截断
2. 检查Prompt设计
    - 是否明确要求基于检索内容回答
    - 是否提供了足够的上下文
3. 检查模型参数
    - Temperature是否过高

        - 是否使用了合适的模型
4. 增加后处理
        - 事实核查步骤
        - 引用标注要求

Q59: 为什么要引入父子索引？BM25引入的原因是什么，比例是怎么设置的，整个具体流程是怎样的，有没有做rerank操作？

解答：

父子索引：

索引类型	作用	大小
子索引	用于精确检索	小（如256 tokens）
父索引	用于生成上下文	大（如2048 tokens）

BM25引入原因：

- 向量检索擅长语义匹配
- BM25擅长精确关键词匹配
- 两者互补提高召回率

混合比例：

- 通常向量检索:关键词检索 = 70%:30%
- 根据场景调整

流程：

```
查询 → 向量化 + 分词 → 向量检索 + BM25检索 → 结果融合 → Rerank → Top-K
```

Q60: 如果做了rerank，rerank之后返回几个块?

解答:

返回数量考虑因素:

因素	建议
上下文窗口	预留足够空间给生成
块大小	大块少取，小块多取
相关性分布	取相关性差距明显的Top-K

典型值:

- 小块（256 tokens）：5-10个
- 大块（1024 tokens）：2-3个

Q61: rerank之后的topk截断是怎么实现的？为什么选这个k值，有没有考虑过其他方案?

解答:

截断方法:

1. 固定数量：直接取Top-K
2. 阈值截断：取相关性>阈值的
3. 动态截断：根据相关性分布动态决定

K值选择考虑:

- 上下文窗口限制
- 信息密度
- 计算成本

其他方案:

- 基于累积相关性的自适应截断
- 基于信息熵的截断

Q62: 讲一下上下文工程，还有记忆功能是怎么实现的

解答：

上下文工程：

组织和管理输入给模型的上下文信息，确保信息质量。

记忆功能实现：

```
短期记忆：内存中维护对话历史
长期记忆：向量数据库存储
        ↓
检索时：查询向量化 → 相似度搜索 → 返回相关记忆
```

Q63: 做RAG项目的时候，是怎么评测效果的？有哪些评测维度，具体用到了哪些指标？

解答：

评测维度：

维度	指标
检索质量	Recall@K、MRR、NDCG
生成质量	BLEU、ROUGE、BERTScore
事实准确性	人工评估、F1
用户体验	满意度评分

Q64: 项目里的数据集包含什么内容，数据来源、数据格式是怎样的？

解答：

（根据具体项目回答）

典型数据：

- 文档：PDF、Word、Markdown
- 结构化数据：数据库表
- 问答对：历史问答记录

数据格式：

```
{
  "id": "doc_001",
  "content": "文档内容",
  "metadata": {
    "source": "来源",
    "category": "分类",
    "timestamp": "2024-01-01"
  }
}
```

Q65: 如果让对RAG的相关度和回答效果做优化，有什么思路？有没有更体系化的优化方案？

解答：

优化思路：

阶段	优化点
数据预处理	文档清洗、分块策略优化
Embedding	领域微调、多向量表示
检索	混合检索、查询改写
重排序	训练领域专用reranker
生成	Prompt优化、引用标注

体系化方案：

- 建立评估流水线
- A/B测试不同策略
- 持续监控和迭代

Q66: 有一千条数据，需要做求和处理，这种数据处理场景怎么设计实现？

解答：

设计方案：

```
数据分片 → 并行处理 → 局部聚合 → 全局聚合
```

技术选型：

- 小数据：Pandas
- 大数据：Spark/Flink
- 实时：流处理

Q67: RAG的性能怎么提升，有没有实际的优化思路（工程层面或算法层面）？

解答：

工程层面：

优化点	方案
索引优化	HNSW、IVF等近似最近邻算法
缓存	热门查询结果缓存
异步化	检索和生成并行
批量处理	批量Embedding

算法层面：

- 查询重写
- 多路召回
- 重排序优化

Q68: 项目里的上下文是怎么处理的？上下文过长、冗余等问题有什么优化方向？

解答：

处理方式：

1. 去重：移除重复信息
2. 压缩：摘要长文档
3. 重排序：按重要性排序
4. 截断：保留最相关部分

Q69: 最近有哪些比较新的RAG框架？例如GraphRAG等

解答：

新框架：

框架	特点
GraphRAG	基于知识图谱的RAG
Self-RAG	自适应检索决策
Corrective RAG	检索结果纠错
RAPTOR	树状检索结构
LightRAG	轻量级高效RAG

Q70: MinerU解析出的Markdown如果遇到跨页的复杂长表格，如何做语义合并的？

解答：

处理方案：

1. 表格识别
    - 检测表格边界
    - 识别表头和表尾
2. 跨页合并
    - 通过表头匹配识别同一表格
    - 合并跨页内容
3. 语义恢复
    - 重建表格结构
    - 保留单元格关系

Q71: 你的混合检索里，BM25是在全文上做还是在MinerU提取的特定Key-Value字段上做？

解答：

两种方案对比：

方案	优点	缺点
全文BM25	信息完整	噪声多
字段BM25	精度高	可能遗漏信息

推荐方案：

- 全文BM25 + 字段权重提升
- 对关键字段（标题、关键词）给予更高权重

Q72: Ragas评测中，如果检索精度高但答案相关性低，最可能的瓶颈在哪?

解答：

可能瓶颈：

1. 上下文整合问题
    - 检索到的内容没有被有效利用
    - Prompt设计不合理
2. 生成阶段问题
    - 模型过度依赖自身知识
    - Temperature过高导致发散
3. 后处理问题
    - 缺少引用约束
    - 没有要求基于检索内容回答

Q73: 在RAG中，递归字符切片相比固定长度切片优势在哪?

解答：

切片方式	优势
固定长度	简单、均匀
递归字符	保持语义完整性、不切断句子

递归字符切片特点：

- 优先按段落切分
- 段落内按句子切分
- 保持自然语言边界

Q74: 如果要实现只在特定分类的文档中进行检索，在Milvus中是使用多个Collection效率高，还是使用分区效率高？

解答：

对比：

方案	适用场景	效率
多Collection	分类差异大、数据隔离要求高	高（物理隔离）
分区（**Partition**）	分类相关、需要跨分类查询	高（逻辑隔离）

推荐：

- 分类数量少：分区
- 分类数量多、数据量大：多Collection

Q75: chunk大小怎么确定？不同业务场景的策略差异？

解答：

影响因素：

因素	建议
文档类型	技术文档大chunk，FAQ小chunk
Embedding模型	不超过模型最大输入
检索粒度	细粒度用小chunk

场景差异：

场景	CHUNK大小	原因
客服FAQ	128-256	问题简短
技术文档	512-1024	需要完整上下文
法律合同	1024+	条款关联性强

Q76: 检索结果相关性不高，怎么优化召回？

解答：

优化策略：

1. 查询优化
    - 查询改写
    - 查询扩展
2. 索引优化
    - 领域微调Embedding
    - 混合检索
3. 后处理
    - 重排序
    - 结果过滤

Q77: 工业图纸识别如果大模型出现了幻觉，你在Prompt层面或后处理层面有什么方法？

解答：

Prompt层面：

- 要求模型只描述可见内容
- 添加约束："如果不确定，请说明"

- 提供示例说明期望输出

后处理层面：

- 与原始图像对比验证
- 规则校验（尺寸、比例等）
- 人工审核流程

三、Multi-Agent多智能体系统

Q78: multi agent架构主流的实现方案

解答：

（同Q10）

Q79: multi agent的交流是靠什么

解答：

（同Q11）

Q80: 如何搭建多智能体系统？

解答：

（同Q14）

Q81: 常见的多智能体架构有哪些?

解答:

（同Q15）

Q82: 多智能体系统设计架构?

解答:

典型架构:

```
┌──────────────────────────────────────────┐
│            协调层（Orchestrator）          │
│       任务分配、冲突解决、状态同步          │
└──────────────────────────────────────────┘
                  │
                  ↓
┌──────────────────────────────────────────┐
│               Agent池                     │
│   ┌──────┐  ┌──────┐  ┌──────┐  ┌──────┐  │
│   │Agent1│  │Agent2│  │Agent3│  │Agent4│  │
│   └──────┘  └──────┘  └──────┘  └──────┘  │
└──────────────────────────────────────────┘
                  │
                  ↓
┌──────────────────────────────────────────┐
│            共享资源层                      │
│       消息队列、状态存储、工具注册中心      │
└──────────────────────────────────────────┘
```

Q83: 上下文管理? 怎么确保多智能体系统并行而不是读同一个上下文?

解答:

上下文隔离方案:

1. 命名空间隔离

        - 每个Agent有独立的上下文命名空间
    2. **会话隔离**
        - 不同会话完全隔离
    3. **只读共享**
        - 共享知识库只读
        - 工作空间独立

Q84: 在A2A场景下，如何防止两个Agent陷入递归对话？

解答：

防护机制：

1. **轮次限制**
    - 设置最大对话轮次
2. **状态检测**
    - 检测对话是否循环
    - 相似度判断
3. **终止条件**
    - 明确定义结束条件
4. **超时机制**
    - 设置对话超时时间

Q85: 了解目前主流的MultiAgent框架吗？如果将你的心理咨询Agent拆分，你认为状态同步的难点在哪？

解答：

主流框架：

- LangGraph、AutoGen、CrewAI、MetaGPT

状态同步难点：

难点	说明
一致性	多个Agent看到的状态一致
实时性	状态变更及时同步
冲突解决	并发修改的处理
性能	同步带来的开销

Q86: 谈谈对A2A通信的理解

解答：

A2A（Agent-to-Agent）是Agent间的标准化通信协议。

核心能力：

- 能力发现：Agent可以发现其他Agent的能力
- 任务委托：一个Agent可以委托任务给另一个
- 结果返回：异步返回执行结果
- 安全认证：确保通信安全

Q87: mult-agent项目架构设计

解答：

（同Q82）

Q88: 局限性与未来优化空间

解答：

当前局限：

- 协调复杂度高
- 调试困难
- 成本高

优化方向：

- 更智能的协调算法
- 更好的可观测性
- 成本优化策略

四、Transformer与注意力机制

Q89: Transformer中Attention的本质是什么？你能从数学角度简要解释一下吗？

解答：

本质： 加权求和，计算序列元素间的相关性。

数学表达：

```
Attention(Q, K, V) = softmax(QK^T / √d_k) × V

其中：
Q = X × W_Q  (Query)
K = X × W_K  (Key)
V = X × W_V  (Value)
```

直观理解：

- Q：我要查询什么
- K：我有什么信息
- V：信息的内容是什么

Q90: 为什么Attention要做缩放?

解答:

原因:

当d_k较大时，QK^T的点积结果会很大，导致softmax梯度消失。

缩放因子**√d_k**的作用:

- 控制点积的数值范围
- 保持方差稳定
- 避免梯度问题

Q91: Self-Attention和Cross-Attention在作用上有什么区别?

解答:

类型	输入	作用
Self-Attention	Q=K=V（同一序列）	学习序列内部关系
Cross-Attention	Q来自一个序列，K/V来自另一个	学习两个序列间关系

应用场景:

- Self-Attention：编码器、解码器自注意力
- Cross-Attention：机器翻译（源语言→目标语言）

Q92: 如果序列长度非常长，Attention计算复杂度问题怎么解决?

解答:

优化方案:

方法	复杂度	原理
稀疏Attention	O(n)	只计算局部注意力
线性Attention	O(n)	核函数近似softmax
FlashAttention	$O(n^2)$	IO感知的分块计算
MQA/GQA	-	共享K/V，减少缓存

Q93: 在Agent多轮对话任务中，你觉得Attention的局限性体现在哪些方面?

解答:

局限性:

1. 上下文长度限制
    - 长对话超出上下文窗口
2. 位置偏见
    - 对长距离依赖建模能力有限
3. 计算成本高
    - 长序列计算量大
4. 信息遗忘
    - 早期信息可能被稀释

Q94: 在长上下文对话里，模型为什么容易出现信息遗忘?

解答:

原因:

1. **Softmax稀释**
    - 注意力权重被大量token分摊
2. 位置编码限制
    - 长距离位置信息衰减

3. 训练数据偏差
    - 训练时短序列为主

Q95: 目前常见有哪些机制可以缓解这种问题?

解答:

缓解机制:

机制	说明
RAG	外部检索补充信息
记忆模块	显式存储关键信息
摘要压缩	定期压缩历史对话
Long Context	使用长上下文模型
Hierarchical Attention	分层注意力机制

五、大模型训练与微调

Q96: 简要介绍一下SFT的核心流程以及数据集的构建策略

解答:

SFT流程:

```
预训练模型 → 准备指令数据 → 监督微调 → 评估 → 部署
```

数据构建策略:

策略	说明
多样性	覆盖多种任务类型
质量优先	人工审核确保准确性
难度分布	简单:中等:困难 = 3:5:2
格式统一	统一指令-回答格式

Q97: 在数据构建时，如何保证数据质量和多样性?

解答:

质量保证:

- 人工标注+审核
- 自动过滤低质量数据
- 去重处理

多样性保证:

- 多来源数据采集
- 数据增强（改写、扩展）
- 任务类型覆盖

Q98: SFT之后常见的Post-Training还有哪些？它们之间的目的有何区别?

解答:

阶段	目的	方法
SFT	学习指令遵循能力	监督学习
RLHF	对齐人类偏好	PPO+奖励模型
DPO	直接偏好优化	对比学习
RLAIF	AI反馈强化学习	AI评判替代人类

Q99: 在什么情况下需要引入RLHF或DPO?

解答:

需要引入的情况:

1. **SFT**后模型输出质量不稳定
2. 需要精细控制输出风格
3. 存在多个合理答案需要排序
4. 需要减少有害输出

RLHF vs DPO选择:

- 追求最佳效果且有资源:RLHF
- 快速迭代、资源有限:DPO

Q100: PPO和DPO在大模型对齐中的主要区别是什么?

解答:

维度	**PPO**	**DPO**
流程	训练奖励模型→PPO优化	直接优化
模型数量	4个(策略、参考、奖励、价值)	2个(策略、参考)
稳定性	较不稳定	较稳定
计算成本	高	低
超参数	多且敏感	较少

Q101: 为什么一些团队会从PPO转向DPO?

解答:

转向原因:

1. 实现简单
    - 无需训练奖励模型
    - 代码量少
2. 训练稳定
    - 不容易出现崩溃
    - 超参不敏感
3. 效果接近
    - 在很多任务上效果相当
4. 资源友好
    - 计算资源需求低

Q102: DPO训练通常有哪些注意事项?

解答:

注意事项:

1. 数据质量
    - 偏好对要准确
    - 避免矛盾偏好
2. 参考模型
    - 保持冻结
    - 用于计算KL散度
3. 超参数
    - β参数控制偏离程度
    - 学习率不宜过大
4. 评估
    - 使用独立测试集
    - 监控KL散度

Q103: 是否了解GRPO？它解决了什么问题？

解答：

GRPO（Group Relative Policy Optimization）：DeepSeek提出的强化学习算法。

解决的问题：

问题	解决方案
PPO需要价值模型	使用组内奖励基线替代
训练不稳定	简化目标函数
计算成本高	减少模型数量

核心创新：

- 组内采样多次，用组内平均作为基线
- 无需单独训练价值网络

Q104: 项目中微调Qwen时，选择的训练阶段是如何确定的？

解答：

训练阶段选择：

阶段	适用情况
继续预训练	领域数据量大、通用能力需保持
SFT	有高质量指令数据
DPO	需要偏好对齐

确定依据：

- 数据量和质量
- 目标能力
- 资源预算

Q105: Loss函数是如何设计或选择的?

解答:

不同阶段**Loss**:

阶段	**LOSS**函数
SFT	交叉熵损失
DPO	对比损失 + KL约束
PPO	策略梯度 + 价值损失 + KL约束

Q106: 是否尝试过不同微调策略进行对比?

解答:

对比维度:

维度	对比内容
全参数微调 **vs LoRA**	效果vs效率
不同学习率	收敛速度vs效果
不同数据量	数据效率
SFT vs DPO	对齐效果

六、Prompt工程与上下文工程

Q107: 如何设计一个更好的Prompt（提示词）?

解答:

设计原则:

原则	说明
明确性	清晰表达任务要求
具体性	提供具体示例
结构化	使用标记分隔不同部分
约束性	明确输出格式和限制

优化技巧：

- 使用Few-shot示例
- 添加角色设定
- 使用分隔符（###、"""）
- 指定输出格式

Q108: Prompt Engineering和Context Engineering有什么区别？

解答：

维度	PROMPT ENGINEERING	CONTEXT ENGINEERING
关注点	如何设计指令	如何组织上下文
核心问题	"怎么问"	"给什么信息"
技术手段	指令模板、示例选择	信息筛选、检索、压缩
优化目标	让模型理解任务	让模型有充足相关信息

Q109: Prompt自动推荐模块用了哪些优化策略？

解答：

优化策略：

1. **用户意图分析**
    - 分析历史行为
    - 识别任务类型

2. **模板匹配**
    - 基于任务类型推荐模板
    - 相似任务复用
3. **A/B测试**
    - 对比不同Prompt效果
    - 持续优化

Q110: 是否尝试过Prompt压缩?

解答:

压缩方法:

方法	说明
摘要压缩	用LLM生成摘要
关键词提取	保留核心关键词
向量化	用向量表示替代文本
选择性保留	只保留关键句子

Q111: 有没有使用embedding表示或相似度检索来优化Prompt?

解答:

应用场景:

1. **示例选择**
    - 根据查询相似度选择Few-shot示例
2. **Prompt组装**
    - 从Prompt库检索相关模板
3. **上下文检索**
    - 检索相关背景信息

七、工具调用与Function Calling

Q112: Function Calling的可靠性如何保障？如果模型一直调用同一个错误工具怎么办？

解答：

可靠性保障：

层级	措施
参数校验	JSON Schema验证
执行保护	沙箱执行、超时控制
结果验证	检查返回结果格式
重试机制	失败时重试或降级

错误工具处理：

1. 检测机制
    - 监控工具调用频率
    - 识别异常模式
2. 干预措施
    - 暂时禁用问题工具
    - 强制选择其他工具
    - 人工介入
3. 学习改进
    - 分析错误原因
    - 优化工具描述

Q113: 工具的描述（description）怎么写才能让模型准确调用？给出淘天场景的示例

解答：

描述编写原则：

1. 清晰说明功能
2. 明确输入参数
3. 提供使用示例
4. 说明适用场景

淘天场景示例：

```
工具名：search_product
描述：
在天猫/淘宝平台搜索商品。当用户想要购买商品、
比较价格或了解商品信息时使用。

参数：
- keyword：搜索关键词（必需）
- category：商品分类（可选）
- price_range：价格范围（可选）

示例：
用户说"我想买苹果手机"
→ 调用：search_product(keyword="苹果手机")
```

Q114: 介绍一下Function Call原理，模型生成的JSON如何通过逻辑触发表层代码执行并返回给模型？

解答：

原理流程：

```
1. 模型分析用户需求
2. 判断需要调用工具
3. 生成工具调用JSON
4. 系统解析JSON
5. 执行对应函数
6. 获取执行结果
7. 将结果返回给模型
8. 模型生成最终回答
```

代码执行:

```
模型输出 → JSON解析 → 函数映射 → 参数校验 →
函数执行 → 结果封装 → 返回模型
```

Q115: 项目提到了多个工具调用链路，调度策略是如何设计的?

解答:

调度策略:

策略	说明
串行执行	有依赖关系的工具按序执行
并行执行	无依赖的工具并行执行
条件执行	根据前置结果决定是否执行
循环执行	需要迭代时循环调用

Q116: 不同工具之间的调用顺序是如何确定的?

解答:

确定方式:

1. 依赖分析
    - 分析工具间的数据依赖

        - 构建依赖图
2. 拓扑排序
        - 按依赖关系排序
        - 确定执行顺序
3. 动态规划
        - 模型根据中间结果决定下一步

Q117: 是否设计过异常fallback策略？

解答：

Fallback策略：

异常类型	处理策略
工具超时	使用缓存结果或默认值
工具失败	切换到备用工具
参数错误	请求用户澄清
权限不足	引导用户授权

Q118: 如果某个工具调用失败，系统如何继续完成任务？

解答：

处理流程：

```
工具调用失败
      ↓
错误分类（可恢复/不可恢复）
      ↓
可恢复 → 重试/降级/替代方案
      ↓
不可恢复 → 告知用户/转人工
```

八、向量数据库与知识库

Q119: 向量数据库选型对比（Milvus/Pinecone/Chroma），淘天场景怎么选？

解答：

对比：

特性	MILVUS	PINECONE	CHROMA
部署方式	自托管/云	全托管	嵌入式
扩展性	强	强	弱
功能丰富度	高	中	低
成本	中	高	低

淘天场景选择：

- 大规模生产环境：Milvus（开源、可控、功能全）
- 快速原型：Chroma（简单、易用）
- 无运维资源：Pinecone（托管、省心）

Q120: 如何评估一个RAG系统是否有效？

解答：

（同Q57）

Q121: 查询知识库的工具函数，其标准输入和输出是什么？

解答：

（同Q30）

九、后端架构与中间件

Q122: 分布式令牌桶限流、漏桶限流，还有滑动窗口算法限流的原理?

解答:

令牌桶:

```
以固定速率产生令牌放入桶中
请求需要获取令牌才能执行
桶满时丢弃令牌
```

漏桶:

```
请求先进入桶中排队
以固定速率处理请求
桶满时丢弃新请求
```

滑动窗口:

```
将时间划分为小窗口
统计当前窗口内的请求数
超过阈值则限流
```

对比:

算法	突发流量	平滑性	实现复杂度
令牌桶	允许	较好	中
漏桶	不允许	最好	中
滑动窗口	不允许	一般	低

Q123: 滑动窗口的数据结构会包含哪些字段？滑动窗口对比令牌桶有什么缺点？

解答：

数据结构：

```
{
  "window_size": 60,         // 窗口大小（秒）
  "window_granularity": 1,   // 粒度（秒）
  "windows": [               // 各窗口计数
    {"timestamp": 1700000000, "count": 100},
    {"timestamp": 1700000001, "count": 120}
  ],
  "total_limit": 1000        // 总限流阈值
}
```

滑动窗口缺点：

1. **内存占用**：需要存储多个窗口数据
2. **精度问题**：粒度太粗不够精确，太细内存大
3. **突发处理**：无法应对窗口边界的突发流量

Q124: 用Redis的什么数据结构能实现滑动窗口？

解答：

方案：

1. **Sorted Set（ZSET）**
    - Score：时间戳
    - Member：请求ID
    - 使用ZREMRANGEBYSCORE移除过期数据
2. **List + Lua脚本**
    - LPUSH新请求

        - LTRIM保留窗口内数据
    3. **Redis Cell模块**
        - 原生支持限流

Q125: 布隆过滤器的原理和应用场景?

解答:

原理:

使用多个哈希函数将元素映射到bit数组中。

特性:

- 可能存在假阳性（误报）
- 不存在假阴性（漏报）
- 不能删除元素

应用场景:

场景	用途
缓存穿透防护	快速判断key是否存在
URL去重	爬虫已访问URL判断
推荐去重	已推荐内容过滤

Q126: 假设一个Agent推理链路包含3个工具调用，并且请求频率较高，系统整体延迟较高，你会如何优化?

解答:

（同Q77）

Q127: 哪些环节最有可能成为性能瓶颈?

解答:

环节	瓶颈原因
模型推理	计算密集、耗时长
工具调用	网络IO、外部依赖
向量检索	大规模数据检索
序列化	大量数据传输

Q128: 在保证准确率的前提下，是否有办法减少调用链路?

解答:

优化方案:

1. 工具合并
    - 将相关工具合并为复合工具
2. 缓存优化
    - 缓存工具调用结果
3. 预计算
    - 预计算常用结果
4. 模型优化
    - 使用更快的模型处理简单任务

Q129: SSE在前后端是如何交互的?

解答:

交互流程:

```
前端                        后端
  |                          |
  |---- HTTP请求 -------->|
  |     (Accept: text/event-stream)
  |                          |
  |<--- 建立连接 ---------|
  |                          |
  |<--- data: {...}\n\n --|  (持续推送)
  |<--- data: {...}\n\n --|
  |                          |
  |---- 关闭连接 -------->|
```

特点:

- 单向推送（服务器→客户端）
- 基于HTTP
- 自动重连

Q130: 后端以什么数据格式推送流式信息?

解答:

SSE格式:

```
data: {"token": "Hello", "index": 0}\n\n
data: {"token": " World", "index": 1}\n\n
event: done\ndata: [DONE]\n\n
```

字段说明:

- `data:`: 数据内容
- `event:`: 事件类型
- `id:`: 事件ID
- `retry:`: 重连时间

Q131: 在使用FastAPI开发大模型接口时，中间件和依赖注入分别适合处理什么样的业务逻辑?

解答:

中间件适用场景:

场景	说明
认证鉴权	验证用户身份
日志记录	记录请求响应
限流	控制请求频率
跨域处理	CORS
错误处理	统一异常处理

依赖注入适用场景:

场景	说明
数据库连接	请求级DB会话
配置读取	获取配置项
服务实例	获取服务类实例
上下文信息	获取用户上下文

Q132: Python的multiprocessing和threading你会如何结合使用来提高整体吞吐量?

解答:

结合策略:

```
IO密集型任务 → Threading（网络请求、文件IO）
CPU密集型任务 → Multiprocessing（模型推理、数据处理）
```

架构设计:

```
主进程
  ├─ 进程池（处理CPU密集型）
  │    ├─ 子进程1
  │    └─ 子进程2
  └─ 线程池（处理IO密集型）
       ├─ 线程1
       └─ 线程2
```

Q133: asyncio.gather和asyncio.as_completed在并发请求多个模型接口时有什么区别？如果其中一个接口超时，如何做降级？

解答：

区别：

函数	行为
gather	等待所有任务完成，返回所有结果
as_completed	按完成顺序返回，先完成的先处理

超时降级：

```python
# 使用wait_for设置超时
try:
    result = await asyncio.wait_for(call_api(), timeout=5.0)
except asyncio.TimeoutError:
    result = await fallback_api()  # 降级方案
```

十、数据库

Q134: MySQL索引失效的情况有哪些?

解答:

失效情况:

情况	说明
使用函数	WHERE YEAR(create_time) = 2024
类型转换	字符串字段用数字查询
like '%xx'	前模糊查询
OR条件	部分条件无索引
!= 或 <>	不等于操作
IS NULL	某些情况下失效
索引列运算	WHERE id + 1 = 10

Q135: like查询会不会导致索引失效?

解答:

规则:

写法	是否使用索引
`like 'abc%'`	是（前缀匹配）
`like '%abc'`	否（后缀匹配）
`like '%abc%'`	否（全模糊）

优化方案:

- 使用全文索引
- 使用反向索引
- 使用搜索引擎（ES）

Q136: MySQL的事务隔离级别，还有一致性相关内容?

解答:

隔离级别:

级别	脏读	不可重复读	幻读
READ UNCOMMITTED	✓	✓	✓
READ COMMITTED	✗	✓	✓
REPEATABLE READ	✗	✗	✓
SERIALIZABLE	✗	✗	✗

一致性:

- ACID中的C（Consistency）
- 事务执行前后数据库处于一致状态

Q137: MVCC的原理? 具体场景下会创建几个readview?

解答:

原理:

通过保存数据的历史版本（undo log），实现非阻塞读。

ReadView创建时机:

隔离级别	**READVIEW创建**
READ COMMITTED	每条SELECT创建一个
REPEATABLE READ	事务开始时创建一个

Q138: MySQL的锁相关知识（行锁、表锁等）？

解答：

锁类型：

锁	粒度	场景
表锁	整张表	DDL操作、MyISAM
行锁	单行	DML操作、InnoDB
间隙锁	索引间隙	防止幻读
临键锁	行+间隙	默认行锁类型
意向锁	表级	表示事务意图

Q139: mysql/pgsql区别？数据结构？

解答：

区别：

特性	MYSQL	POSTGRESQL
数据类型	基础类型丰富	支持数组、JSON、自定义类型
扩展性	插件式	高度可扩展
复杂查询	一般	强大（CTE、窗口函数）
并发控制	MVCC	MVCC（实现更优）
GIS	支持	更强大的PostGIS

十一、缓存与Redis

Q140: 讲一下LRU的原理和实现

解答：

原理：

淘汰最近最少使用的数据。

实现思路：

```
使用哈希表 + 双向链表
- 哈希表：O(1)查找
- 双向链表：O(1)移动和删除

访问数据：移到链表头部
插入数据：加到链表头部
淘汰数据：删除链表尾部
```

Q141: Redis内存竞争？分布式锁？

解答：

内存竞争：

问题	解决方案
内存不足	设置maxmemory、淘汰策略
大Key	拆分、压缩
热Key	本地缓存、读写分离

分布式锁：

```
SET key value NX EX 30  # 获取锁
DEL key                 # 释放锁
```

Redisson实现：

- 可重入锁
- 看门狗自动续期
- 红锁（多节点）

Q142: Redis缓存策略？更新策略？

解答：

缓存策略：

策略	说明
Cache Aside	应用管理缓存，常用
Read Through	缓存未命中时自动加载
Write Through	写操作同步更新缓存
Write Behind	异步更新缓存

更新策略：

策略	一致性	性能
先删缓存再更新DB	低	高
先更新DB再删缓存	中	高
延迟双删	较高	中
Canal订阅binlog	高	低

Q143: 为什么用Stream？

解答：

Redis Stream优势：

特性	说明
持久化	数据不丢失
消费者组	支持多消费者
消息确认	ACK机制
消息追溯	可读取历史消息
阻塞读取	支持阻塞等待

适用场景：

- 消息队列
- 事件流处理
- 日志收集

Q144: 更新策略？Redis旧数据库新咋办？

解答：

（同Q142）

Q145: 缓存击穿/穿透/雪崩？

解答：

问题	现象	解决方案
缓存穿透	查询不存在数据，打到DB	布隆过滤器、缓存空值
缓存击穿	热点Key过期，大量请求打到DB	互斥锁、逻辑过期
缓存雪崩	大量Key同时过期	随机过期时间、多级缓存

Q146: redis的持久化机制有哪些?

解答:

机制	说明	特点
RDB	快照持久化	文件小、恢复快、可能丢数据
AOF	日志持久化	数据安全、文件大、恢复慢
混合	RDB+AOF	兼顾两者优点

Q147: 手撕：实现LRU缓存机制

解答:

核心思路:

使用哈希表 + 双向链表实现O(1)操作。

十二、算法与数据结构

Q148: 手撕 判断有效括号

解答:

思路:

使用栈结构，遇到左括号入栈，遇到右括号与栈顶匹配。

复杂度:

- 时间：O(n)
- 空间：O(n)

Q149: 手撕 反转链表

解答：

思路：

迭代法：遍历链表，逐个反转指针方向。

复杂度：

- 时间：O(n)
- 空间：O(1)

Q150: 手撕 全排列

解答：

思路：

回溯法：递归交换元素位置，生成所有排列。

复杂度：

- 时间：O(n!)
- 空间：O(n)

Q151: 手撕 岛屿数量

解答：

思路：

DFS/BFS：遍历网格，遇到1则进行搜索标记连通区域。

复杂度：

- 时间：O(m×n)
- 空间：O(m×n)

Q152: 算法题：岛屿数量（追问）时间复杂度和空间复杂度是多少？如果数据规模非常大，有没有优化思路？

解答：

复杂度：

- 时间：O(m×n)
- 空间：O(m×n)（递归栈或队列）

大数据优化：

方案	说明
并查集	适合流式处理
分治	分块处理再合并
分布式	MapReduce处理

Q153: 手撕 旋转数组找最小值

解答：

思路：

二分查找：利用数组部分有序特性。

复杂度：

- 时间：O(log n)
- 空间：O(1)

Q154: 手撕 统计字符出现频率并找TopK

解答：

思路：

1. 哈希表统计频率
2. 最小堆/快速选择找TopK

复杂度：

- 时间：O(n log k)
- 空间：O(n)

Q155: 手撕 螺旋矩阵

解答：

思路：

按层模拟：顺时针遍历矩阵的每一层。

复杂度：

- 时间：O(m×n)
- 空间：O(1)（不考虑输出）

Q156: 手撕 两两交换链表中的节点

解答：

思路：

迭代法：每两个节点为一组，交换后连接下一组。

复杂度：

- 时间：O(n)
- 空间：O(1)

Q157: 手撕：LeetCode 768

解答：

题目：最多能完成排序的块 II

思路：

使用单调栈，维护每个块的最大值。

Q158: SQL：统计task表中每个node的成功率

解答：

```sql
SELECT
    node,
    COUNT(CASE WHEN status = 'success' THEN 1 END) * 1.0 / COUNT(*)
AS success_rate
FROM task
GROUP BY node;
```

Q159: 算法：有放回组合问题，n个元素中选k个组合数

解答：

思路：

回溯法：每个位置可以选择任意元素（可重复）。

复杂度：

- 时间：O(n^k)
- 空间：O(k)

Q160: 怎么优化递归

解答：

优化方法：

方法	说明
记忆化	缓存子问题结果
动态规划	自底向上计算
尾递归	编译器优化
迭代替代	用循环替代递归

十三、Python与Go语言

Q161: 讲讲python的gil（全局解释器锁）是什么？

解答：

GIL（Global Interpreter Lock）：

- Python解释器的全局锁
- 保证同一时刻只有一个线程执行Python字节码
- 多线程无法真正并行执行CPU密集型任务

影响：

- IO密集型：多线程有效
- CPU密集型：多线程无效，需用多进程

Q162: 那如果要实现真正的并行计算，python里有什么方案?

解答:

方案:

方案	适用场景
multiprocessing	CPU密集型
concurrent.futures	统一接口
C扩展	释放GIL
Numba/JIT	数值计算
其他解释器	Jython、IronPython

Q163: Go语言的GMP调度模型是怎样的?

解答:

GMP模型:

组件	说明
G（Goroutine）	协程，用户态轻量级线程
M（Machine）	系统线程，执行Goroutine
P（Processor）	逻辑处理器，维护Goroutine队列

调度流程:

```
Goroutine创建 → 放入P的本地队列 → M获取G执行
                        ↓
              本地队列满 → 放入全局队列
```

Q164: GMP模型中的Machine（M）代表什么？

解答：

M（Machine）：

- 操作系统线程（OS Thread）
- 负责实际执行Goroutine的代码
- M与P绑定后才能执行G
- 数量通常与CPU核心数相关

Q165: Processor（P）上的本地队列长度通常是多少？

解答：

默认长度：

- 本地队列：256个Goroutine
- 全局队列：无限制

设计原因：

- 避免频繁访问全局队列（需要锁）
- 提高调度效率

Q166: 讲讲langchain的核心组件有哪些？

解答：

核心组件：

组件	功能
Models	模型接口（LLM、ChatModel）
Prompts	提示词管理
Indexes	文档索引和检索
Chains	组件链式组合
Agents	智能体
Memory	记忆管理
Tools	工具集成

十四、计算机网络

Q167: 讲讲tcp的三次握手

解答：

三次握手过程：

```
客户端                          服务器
  |                               |
  |---- SYN,seq=x --------->|   (第一次：客户端请求建立连接)
  |                               |
  |<--- SYN,seq=y,ACK=x+1 --|   (第二次：服务器确认并请求)
  |                               |
  |---- ACK=y+1 ----------->|   (第三次：客户端确认)
  |                               |
  |     连接建立，开始传输     |
```

为什么是三次：

- 防止历史重复连接
- 同步双方初始序列号

- 确认双方收发能力

十五、其他技术问题

Q168: 为什么选择Spring AI框架?

解答:

优势:

优势	说明
生态集成	与Spring生态无缝集成
抽象统一	统一不同AI提供商接口
配置驱动	通过配置切换模型
企业级特性	监控、安全、可观测性

Q169: Spring AI框架的主要优势是什么?

解答:

（同Q168）

Q170: 如何设计多模型支持架构?

解答:

架构设计:

```
┌──────────────────────────────────────────┐
│              统一接口层                    │
│     generate() / chat() / embed()         │
```

```
└──────────────────────────────────┘
                  │
                  ↓
┌──────────────────────────────────┐
│            模型适配层            │
│  OpenAIAdapter │ ClaudeAdapter │ ...  │
└──────────────────────────────────┘
                  │
                  ↓
┌──────────────────────────────────┐
│            底层模型              │
│    GPT-4   Claude   Qwen   ...   │
└──────────────────────────────────┘
```

Q171: 多租户环境下模型切换是否支持热更新？切换是否相互独立？

解答：

热更新支持：

- 配置中心动态推送
- 模型连接池动态管理
- 无感知切换

独立性：

- 每个租户独立配置
- 模型实例隔离
- 资源配额独立

Q172: 讲一下CNN/RNN/LSTM？

解答：

模型	特点	适用场景
CNN	局部连接、权值共享	图像处理
RNN	序列建模、有记忆	文本序列
LSTM	解决RNN梯度消失	长序列建模

Q173: NLP生成时如何选择最优序列?

解答:

解码策略:

策略	说明
Greedy	每步选概率最高的
Beam Search	保留Top-K候选
Sampling	按概率采样
Top-p	核采样
Top-k	限制候选集

Q174: 你最熟悉的主力编程语言是什么?

解答:

（根据个人情况回答）

Q175: 系统如何实现图像识别等多模态功能?

解答:

实现方案:

```
图像输入 → 预处理 → 多模态模型（CLIP/GPT-4V）→ 特征提取 → 下游任务
```

技术选型：

- 开源：CLIP、LLaVA
- 商业：GPT-4V、Gemini

Q176: 为什么不直接使用多模态大模型？

解答：

考虑因素：

因素	说明
成本	多模态模型更贵
延迟	响应更慢
精度	特定任务专用模型更准
可控性	模块化更易调优

Q177: 在没有前端界面的情况下，本地图片是如何传到后端并进行识别的？

解答：

传输方式：

1. **Base64编码**
    - 图片转Base64字符串
    - 通过JSON传输
2. **Multipart表单**
    - 使用form-data格式
    - 适合大文件
3. **URL传递**
    - 先上传到OSS
    - 传递URL给后端

Q178: 对其他主流AI框架（如LangChain）有了解吗？

解答：

（同Q166）

Q179: 如果数据维度很多，你怎么分析影响因素？

解答：

分析方法：

方法	适用场景
PCA	降维、找主成分
相关性分析	线性关系
特征重要性	树模型输出
SHAP值	解释模型预测
因果推断	找因果关系

Q180: 讲一下vibe coding做前端的策略？

解答：

Vibe Coding策略：

1. 快速原型
    - 用AI生成基础代码
    - 快速验证想法
2. 迭代优化
    - 根据效果调整Prompt
    - 逐步完善功能
3. 人工审核

- 检查代码质量
- 确保安全性和性能

附录：面试手撕代码题目汇总

序号	题目	难度
1	判断有效括号	⭐
2	反转链表	⭐
3	LRU缓存实现	⭐⭐
4	岛屿数量	⭐⭐
5	全排列	⭐⭐
6	旋转数组找最小值	⭐⭐
7	统计字符频率找TopK	⭐⭐
8	螺旋矩阵	⭐⭐⭐
9	两两交换链表节点	⭐⭐
10	LeetCode 768	⭐⭐⭐
11	SQL：统计成功率	⭐⭐
12	有放回组合	⭐⭐

总结

本文档共整理了 **180+** 个大模型技术栈面试问题，涵盖以下15个技术领域：

1. **Agent设计与架构**（45题）
2. **RAG检索增强生成**（24题）
3. **Multi-Agent多智能体系统**（11题）
4. **Transformer与注意力机制**（7题）
5. 大模型训练与微调（11题）
6. **Prompt工程与上下文工程**（5题）

7. **工具调用与Function Calling**（7题）
8. **向量数据库与知识库**（3题）
9. **后端架构与中间件**（12题）
10. **数据库**（6题）
11. **缓存与Redis**（8题）
12. **算法与数据结构**（13题）
13. **Python与Go语言**（6题）
14. **计算机网络**（1题）
15. **其他技术问题**（12题）

文档生成时间：2026-03-21